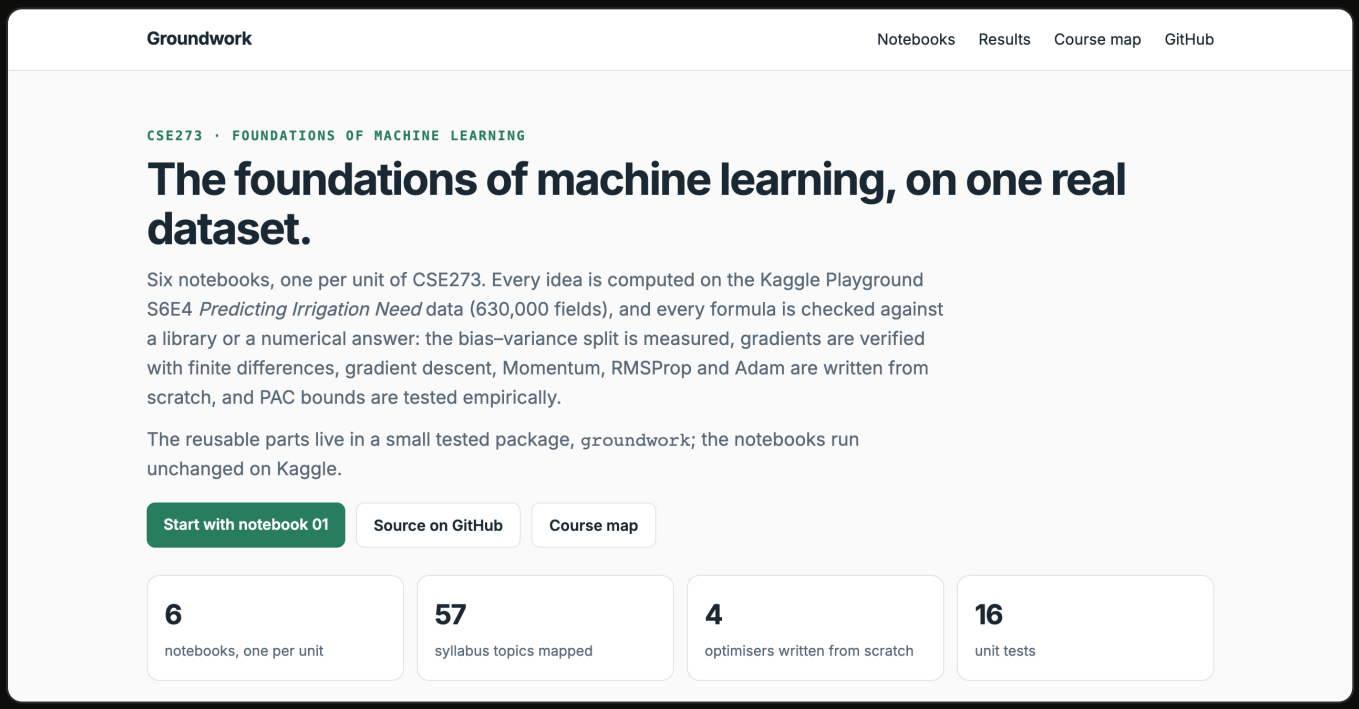
Groundwork

The foundations of machine learning, worked through on one real dataset. Six notebooks, one per unit, run on the 630,000-field Kaggle irrigation competition I entered. Every formula is checked against a library or a numerical answer, the optimisers are written from scratch, and the learning-theory bounds are tested by experiment.



STUDENT Lukshya Singh Supyal
REGISTRATION NO.
PROGRAMME B.Tech CSE, Semester 5
COURSE CSE273 Foundations of Machine Learning
WEBSITE lushadowx.github.io/Groundwork
SOURCE CODE github.com/LuShadowX/Groundwork

CONTENTS

What's in this report

01 **Project overview**
Idea, data, numbers

02 **How it's built**
Package, notebooks, site

03 **Unit I**
ML foundations and NumPy

04 **Unit II**
Pandas and Matplotlib

05 **Unit III**
Exploratory data analysis

06 **Unit IV**
Linear algebra, probability, calculus

07 **Unit V**
Statistics and optimisation

08 **Unit VI**
Statistical learning theory

09 **Syllabus mapping**
All 57 topics

10 **Code walkthrough**
The important functions

11 **Testing**
What was checked

12 **Deployment and conclusion**
Site, Kaggle, what I learned

> Every figure and number in this report comes from executing the notebooks on the real data; the figures are the files the notebooks saved, and the numbers are copied from their outputs. Code extracts are copied from the source files with their file names and line numbers.

Course and scope

The project follows the CSE273 syllabus as published unit by unit on the university notes site: Machine Learning Foundation, Pandas for Data Handling, Exploratory Data Analysis, Linear Algebra and Calculus, Statistics for ML, and Statistical Learning Theory. Each unit has one notebook, and chapter 09 maps all 57 listed topics to the notebook section that covers them. The course page does not list numbered course outcomes, so the mapping is by topic.

Why this dataset

In April 2026 I entered Kaggle's Playground Series S6E4, *Predicting Irrigation Need*, with a quick model (one-hot features, XGBoost, plain argmax), which scored a balanced accuracy of 0.9602 and ranked 3,391 of 4,315. Using the same data for this course makes the foundations concrete: every abstract idea gets tested on something I had already tried to solve. (The applied side, improving that submission, is my CSE274 project, *Irrigate*.)

01 · PROJECT OVERVIEW

Foundations, measured instead of quoted

Foundations courses tend to state results: bias and variance trade off, gradients point uphill, a 95% interval covers the truth 95% of the time, the PAC bound guarantees generalisation. With 630,000 labelled examples, each of those can be measured. Treating the full dataset as the population makes the "true" answer known, so a theorem can be checked as well as stated.

6
notebooks, one per syllabus unit

57
syllabus topics mapped

4
optimisers written from scratch

16
unit tests, all passing

The data

ITEM	VALUE
Source	Kaggle Playground Series S6E4, *Predicting Irrigation Need* (April 2026)
Rows	630,000 training fields (270,000 test fields without labels)
Features	11 numeric (soil pH, moisture, organic carbon, conductivity, temperature, humidity, rainfall, sunlight, wind, field area, previous irrigation) and 8 categorical (soil, crop, growth stage, season, irrigation type, water source, mulching, region)
Target	Irrigation need: Low 58.7%, Medium 37.9%, High 3.3%
Metric	Balanced accuracy (mean of the per-class recalls)

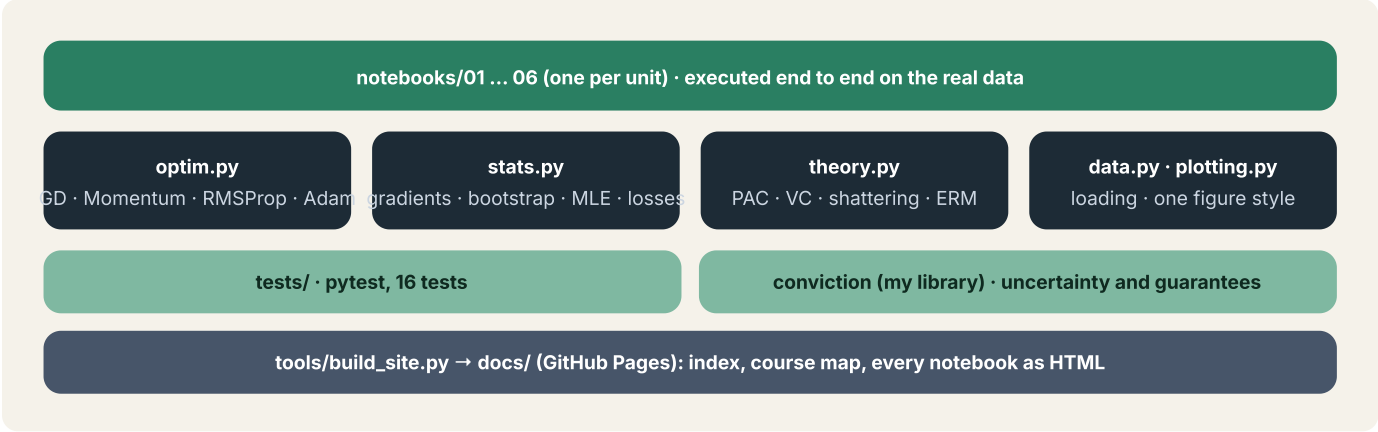
Highlights

- **Bias² and variance measured** by training 30 trees per depth on bootstrap samples: bias² falls from 0.22 to 0.02 while variance rises, and their sum is lowest at depth 6.
- **Gradient descent, Momentum, RMSProp and Adam written from scratch** (about 60 lines). On the irrigation data all four reach within 0.004 of scikit-learn's optimum once each has its own tuned learning rate. On the Rosenbrock valley, Adam and Momentum reach the minimum and plain GD does not.
- **Learning theory tested by experiment.** The PAC bound for 400 decision stumps held in 300 out of 300 samples. My own library `conviction` certified an error rate of at most 0.5%, and on unseen data the rate was 0.37%. The No Free Lunch theorem was checked exhaustively: every learner, including one built to be wrong, averages exactly 50% over all 256 possible targets.

02 · HOW IT'S BUILT

A small package, six notebooks, one site

The parts worth testing live in a Python package; the notebooks import it, explain each idea and run it on the data; a build script turns the executed notebooks into a website. The notebooks run unchanged on Kaggle (the first cell installs the package from GitHub).



Files

FILE	JOB	LINES
`src/groundwork/optim.py`	the four optimisers, Rosenbrock and saddle test functions, softmax regression trained with any optimiser	134
`src/groundwork/stats.py`	numerical gradient, bootstrap CI, normal MLE and negative log-likelihood, loss functions	60
`src/groundwork/theory.py`	PAC and VC sample sizes, linear separability as a linear program, shattering, ERM over stumps	58
`src/groundwork/data.py`	finds and loads the competition data, stratified samples, one-hot matrix	69
`tests/`	16 tests of the package	50 + 51
`notebooks/*.ipynb`	the six units, with all outputs	6 notebooks
`tools/build_site.py`, `tools/syllabus.py`	the website and the course map (also used for chapter 09)	134

02 · HOW IT'S BUILT (CONTINUED)

The notebooks on the website

Each notebook is published with its code, outputs and figures, exactly as executed. These are screenshots of the site.

Groundwork Machine learning foundations and NumPy .ipynb on GitHub

6. Evaluation and performance

Train–test split. The model is scored on fields it never saw. `stratify=y` keeps the 3.3% of High fields in both parts, otherwise a small test set could get very few of them.

Evaluation metrics. Accuracy counts correct predictions. With 59% Low and only 3% High, a model can be very accurate while being bad at High. **Balanced accuracy** averages the recall of each class, so every class counts equally. Precision, recall, F1 and ROC curves are covered in CSE274; here, the per-class recall is enough to see the difference:

In [5]:
```
from sklearn.metrics import recall_score
pd.Series(recall_score(y_test, pred, average=None), index=data.CLASSES, name="recall").round(3)
```

Out[5]:
```
Low       0.993
Medium    0.964
High      0.808
Name: recall, dtype: float64
```

Overfitting and underfitting. A tree that is too shallow can't express the pattern (underfitting: both errors high). A tree that is too deep memorises the noise in the training set (overfitting: training error near 0, test error rising).

In [6]:
```
depths = range(1, 26)
train_err, test_err = [], []
for d in depths:
    m = DecisionTreeClassifier(max_depth=d, random_state=0).fit(X_train, y_train)
    train_err.append(1 - balanced_accuracy_score(y_train, m.predict(X_train)))
    test_err.append(1 - balanced_accuracy_score(y_test, m.predict(X_test)))

best = int(np.argmin(test_err)) + 1
fig, ax = plt.subplots()
ax.plot(depths, train_err, "o-", label="training error", color=plotting.ACCENT)
ax.plot(depths, test_err, "o-", label="test error", color="#e76f51")
ax.axvline(best, ls="--", color="grey")
ax.text(best + 0.3, max(test_err) * 0.9, f"best depth = {best}")
ax.annotate("underfitting", (2, test_err[1]), xytext=(4, max(test_err) * 0.95), arrowprops={"arrowstyle": "->"})
```

Fig. 1 · **Notebook 01**, evaluation and performance: the stratified split and the tree's accuracy against balanced accuracy.

Groundwork Exploratory data analysis .ipynb on GitHub

5. Correlation analysis and multicollinearity

A. Correlation heatmap

In [10]:
```
corr_df = df[num].assign(Need=df[data.TARGET].cat.codes)
corr = corr_df.corr(method="pearson")
fig, ax = plt.subplots(figsize=(10, 8))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r", center=0, vmin=-1, vmax=1,
            annot_kws={"size": 7}, cbar_kws={"shrink": 0.7}, ax=ax)
ax.set_title("Pearson correlation (Need = 0 / 1 / 2)")
plotting.save("g03-heatmap")
plt.show()
corr["Need"].drop("Need").sort_values(key=abs, ascending=False).round(3).to_frame("r with need")
```

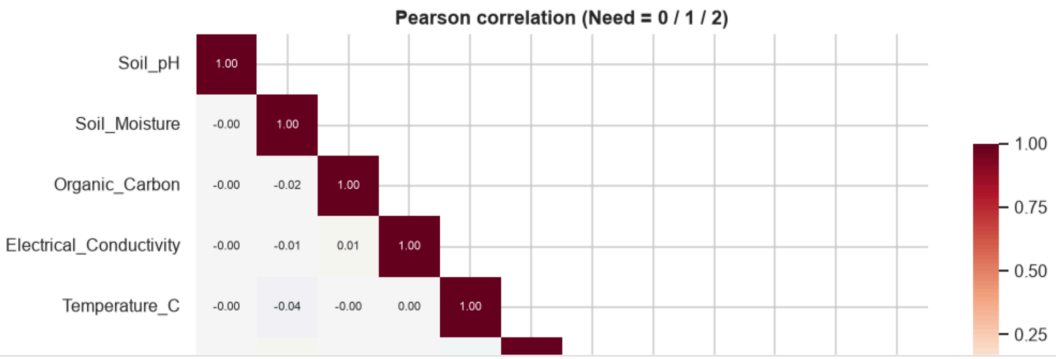


Fig. 2 · **Notebook 03**, correlation analysis: the heatmap code and its output.

Groundwork Linear algebra, probability and calculus .ipynb on GitHub



The arrows (negative gradient) point down the slope towards the low-loss corner: a negative weight on moisture and a positive weight on temperature, which matches the EDA.

4.4 The chain rule

A model is a chain of functions: $u = Xw \to p = \sigma(u) \to L(p)$. The chain rule multiplies the local derivatives: dL/dw = dL/dp · dp/du · du/dw. With σ′(u) = σ(u)(1 − σ(u)), the pieces collapse to the simple gradient above. Back-propagation in neural networks is this same rule applied layer by layer. Here it is done step by step for one field and one weight:

In [14]:
```
i, k = 0, moist
u = Xh[i] @ w0
p = sigmoid(u)
dL_dp = -(yh[i] / p) + (1 - yh[i]) / (1 - p)
dp_du = p * (1 - p)
du_dw = Xh[i, k]
chain = dL_dp * dp_du * du_dw

def single_loss(wk):
    wv = w0.copy(); wv[k] = wk
    q = sigmoid(Xh[i] @ wv)
    return -(yh[i] * np.log(q) + (1 - yh[i]) * np.log(1 - q))

print(f"dL/dp = {dL_dp:.5f}   dp/du = {dp_du:.5f}   du/dw = {du_dw:.5f}")
print(f"chain rule product     = {chain:.6f}")
print(f"simplified (p − y)·x   = {(p - yh[i]) * du_dw:.6f}")
print(f"numerical derivative   = {numerical_grad(lambda v: single_loss(v[0]), np.array([w0[k]]))[0]:.6f}")
```

```
dL/dp = 1.90680   dp/du = 0.24940   du/dw = −0.27489
chain rule product     = −0.130726
simplified (p − y)·x   = −0.130726
numerical derivative   = −0.130726
```

Syllabus topic	Where
Data structures (scalar → tensor)	1.1

Fig. 3 · **Notebook 04**, the chain rule worked step by step, and the unit's syllabus table at the end of the notebook.

Groundwork Statistics for ML and optimisation .ipynb on GitHub

1.3 Confidence intervals

A 95% CI is a range built so that, over repeated samples, 95% of such ranges contain the true value. Three ways for the mean temperature of High fields: the t-formula, the bootstrap from `groundwork.stats`, and `conviction`, my uncertainty library, which carries the bootstrap distribution around as a value.

In [6]:
```
true_high = df.loc[df[data.TARGET] == "High", "Temperature_C"].mean()
m, se = hi_t.mean(), hi_t.std(ddof=1) / np.sqrt(len(hi_t))
t_ci = stats.t.interval(0.95, len(hi_t) - 1, m, se)
_, b_lo, b_hi = bootstrap_ci(hi_t.to_numpy(), n_boot=4000, seed=0)

boot_means = [rng.choice(hi_t.to_numpy(), len(hi_t)).mean() for _ in range(4000)]
belief = cn.empirical(boot_means)          # an Uncertain value
c_lo, c_hi = belief.interval(0.95)

pd.DataFrame({
    "method": ["t-interval", "bootstrap (groundwork)", "conviction.empirical"],
    "low": [t_ci[0], b_lo, c_lo], "high": [t_ci[1], b_hi, c_hi],
}).assign(contains_true_mean=lambda d: (d.low <= true_high) & (true_high <= d.high)).round(3)
```

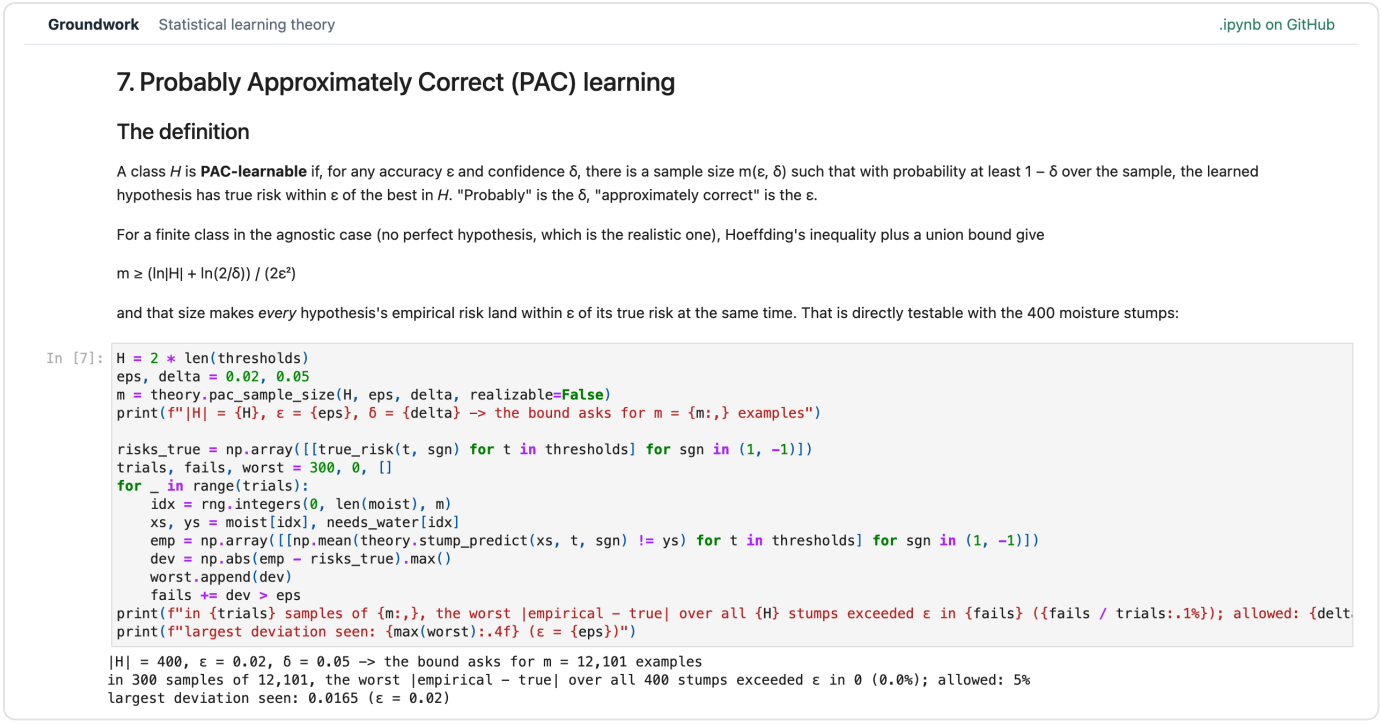
Out[6]:

	method	low	high	contains_true_mean
0	t-interval	33.628	35.526	True
1	bootstrap (groundwork)	33.609	35.465	True
2	conviction.empirical	33.628	35.501	True

`conviction` also answers questions about the uncertain value directly, returning a probability instead of a yes/no. For example, *are High fields at least 9 °C hotter than Low ones?*

In [7]:
```
diff = cn.empirical([rng.choice(hi_t.to_numpy(), len(hi_t)).mean() - rng.choice(lo_t.to_numpy(), len(lo_t)).mean() for _ in range(4000)])
claim = diff > 9.0
print(f"difference: {diff.mean():.2f} °C, 95% interval {np.round(diff.interval(0.95), 2)}")
print(f"P(difference > 9 °C) = {claim.probability():.3f}")
```

Fig. 4 · **Notebook 05**, confidence intervals: t-interval, bootstrap and `conviction` side by side.



Groundwork Statistical learning theory .ipynb on GitHub

7. Probably Approximately Correct (PAC) learning

The definition

A class *H* is **PAC-learnable** if, for any accuracy ε and confidence δ, there is a sample size m(ε, δ) such that with probability at least 1 – δ over the sample, the learned hypothesis has true risk within ε of the best in *H*. "Probably" is the δ, "approximately correct" is the ε.

For a finite class in the agnostic case (no perfect hypothesis, which is the realistic one), Hoeffding's inequality plus a union bound give

$$m \geq (\ln|H| + \ln(2/\delta)) / (2\varepsilon^2)$$

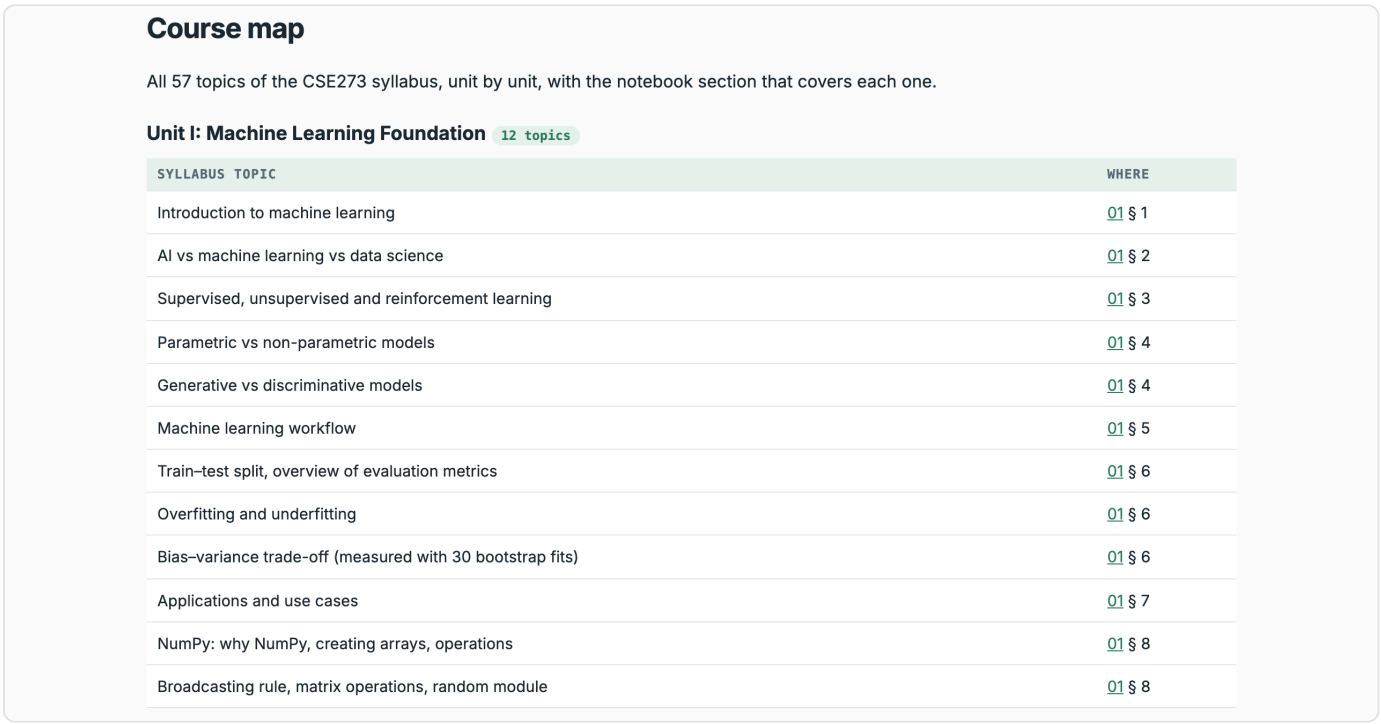
and that size makes *every* hypothesis's empirical risk land within ε of its true risk at the same time. That is directly testable with the 400 moisture stumps:

In [7]:
```
H = 2 * len(thresholds)
eps, delta = 0.02, 0.05
m = theory.pac_sample_size(H, eps, delta, realizable=False)
print(f"|H| = {H}, ε = {eps}, δ = {delta} -> the bound asks for m = {m:,} examples")

risks_true = np.array([[true_risk(t, sgn) for t in thresholds] for sgn in (1, -1)])
trials, fails, worst = 300, 0, []
for _ in range(trials):
    idx = rng.integers(0, len(moist), m)
    xs, ys = moist[idx], needs_water[idx]
    emp = np.array([[np.mean(theory.stump_predict(xs, t, sgn) != ys) for t in thresholds] for sgn in (1, -1)])
    dev = np.abs(emp - risks_true).max()
    worst.append(dev)
    fails += dev > eps
print(f"in {trials} samples of {m:,}, the worst |empirical − true| over all {H} stumps exceeded ε in {fails} ({fails / trials:.1%}); allowed: {delt
print(f"largest deviation seen: {max(worst):.4f} (ε = {eps})")
```

```
|H| = 400, ε = 0.02, δ = 0.05 -> the bound asks for m = 12,101 examples
in 300 samples of 12,101, the worst |empirical − true| over all 400 stumps exceeded ε in 0 (0.0%); allowed: 5%
largest deviation seen: 0.0165 (ε = 0.02)
```

Fig. 5 · **Notebook 06**, PAC learning: the bound and its empirical check.



Course map

All 57 topics of the CSE273 syllabus, unit by unit, with the notebook section that covers each one.

Unit I: Machine Learning Foundation 12 topics

SYLLABUS TOPIC	WHERE
Introduction to machine learning	01 § 1
AI vs machine learning vs data science	01 § 2
Supervised, unsupervised and reinforcement learning	01 § 3
Parametric vs non-parametric models	01 § 4
Generative vs discriminative models	01 § 4
Machine learning workflow	01 § 5
Train–test split, overview of evaluation metrics	01 § 6
Overfitting and underfitting	01 § 6
Bias–variance trade-off (measured with 30 bootstrap fits)	01 § 6
Applications and use cases	01 § 7
NumPy: why NumPy, creating arrays, operations	01 § 8
Broadcasting rule, matrix operations, random module	01 § 8

Fig. 6 · **The course map** on the site: every topic links to its notebook section.

03 · UNIT I

Machine learning foundations and NumPy

Unit I: what machine learning is, AI vs ML vs data science, supervised / unsupervised / reinforcement learning, parametric vs non-parametric and generative vs discriminative models, the workflow, train–test evaluation, overfitting and underfitting, the bias–variance trade-off, applications, and NumPy for numerical computing. Notebook 01

The problem as a learning task

Mitchell's definition fixes the task (classify a field's irrigation need), the experience (630,000 labelled fields) and the performance measure (balanced accuracy). Supervised learning (logistic regression, 72% training accuracy on the numeric features) and unsupervised learning (K-Means with three clusters) were both run on the same data. K-Means never saw the labels, and its clusters only partly match the classes: one cluster holds most High fields, but every cluster mixes Low and Medium.

Model categories, measured

MODEL	CATEGORY	STORED SIZE
Logistic regression	parametric, discriminative	1.0 KB
Gaussian Naive Bayes	parametric, generative	1.1 KB
k-NN (k = 15)	non-parametric	2,240 KB (it stores the training set)

Evaluation, overfitting and the bias–variance trade-off

A depth-8 tree on a stratified 75/25 split reaches 97.6% accuracy but only 92.2% balanced accuracy. Its High recall is 0.81 against 0.99 for Low, which is why balanced accuracy is the right metric for this data.

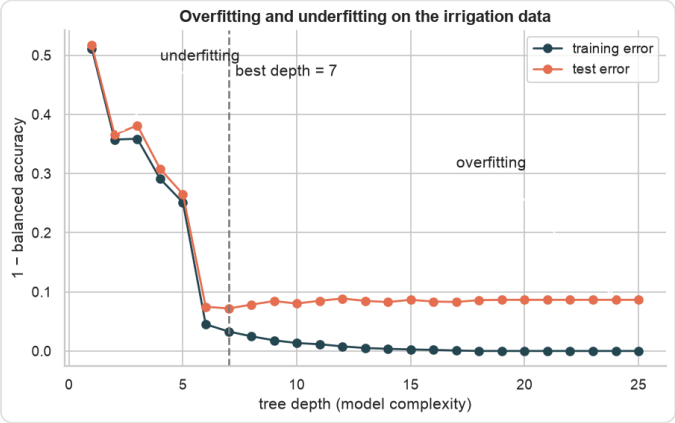


Fig. 7 · **Overfitting and underfitting.** Training error keeps falling with depth; test error is lowest at depth 7.

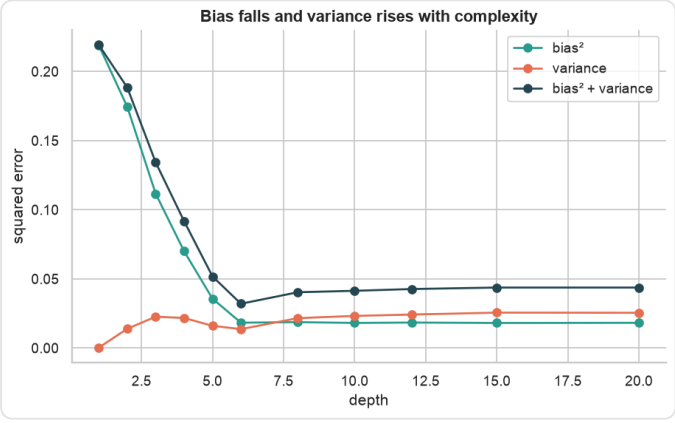


Fig. 8 · **Bias² and variance, measured.** 30 bootstrap trees per depth: bias² falls from 0.22 to 0.02, variance rises, and the sum is lowest at depth 6.

NumPy

The notebook covers creating arrays, element-wise operations, masks and fancy indexing, the broadcasting rule (standardising a 20,000 × 3 matrix in one line, and a shape that can't broadcast), matrix operations (solving a least-squares fit with the normal equation) and the seeded random generator. A vectorised sum over 400,000 values ran 22× faster than the Python loop.

04 · UNIT II

Pandas for data handling

Unit II: Series and DataFrames, reading and writing CSV, inspecting a dataset, `loc` vs `iloc`, boolean filtering, and Matplotlib: line, bar, histogram, scatter and pie charts, customisation and subplots. `Notebook 02`

Reading all 630,000 rows takes 0.24 s. Inspection finds no missing values and no duplicates, and shows that High is 3.3% of fields. Boolean filtering already finds a strong rule: among dry (moisture < 20%), hot (> 35 °C) fields, High is 22.8%, seven times its overall share. Grouping by growth stage and mulching shows that unmulched vegetative and flowering crops are where High lives (10.6–10.8%), compared with under 2% when mulched.

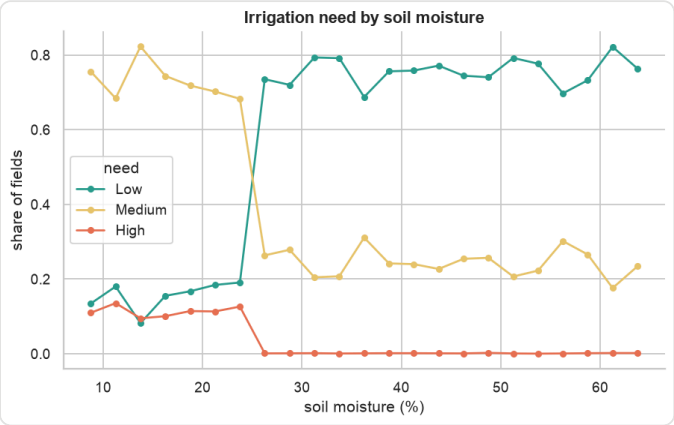


Fig. 9 · **Line plot.** Share of each class across soil-moisture bins: a sharp switch at about 25%.

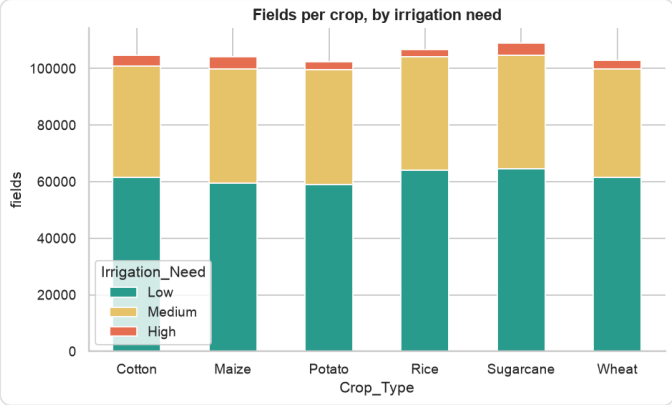


Fig. 10 · **Bar plot.** Fields per crop, stacked by need: crops barely differ.

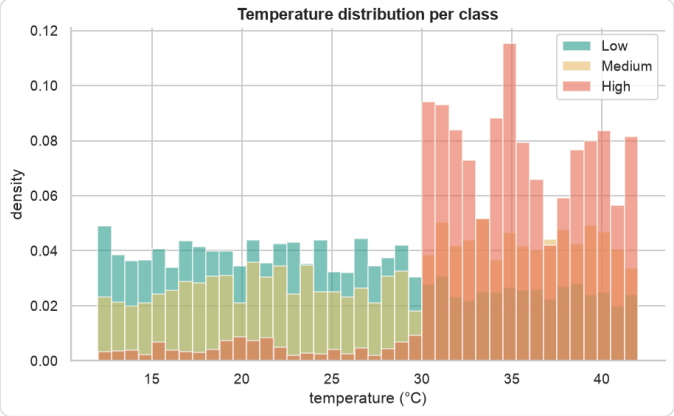


Fig. 11 · **Histogram.** Temperature per class: High fields are concentrated above about 30 °C.

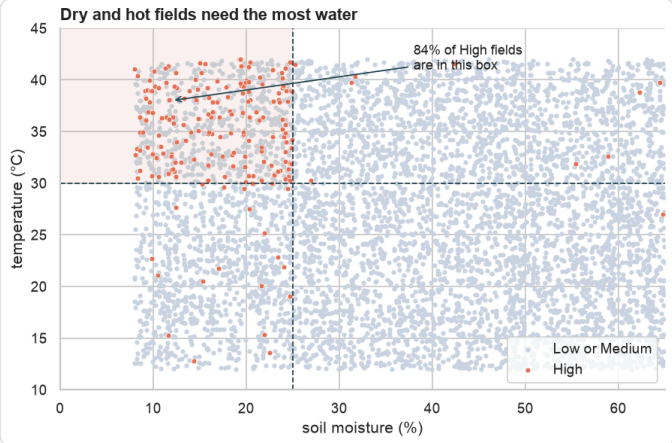


Fig. 12 · **A customised plot** made to make one point: 84% of High fields sit in the dry, hot box.

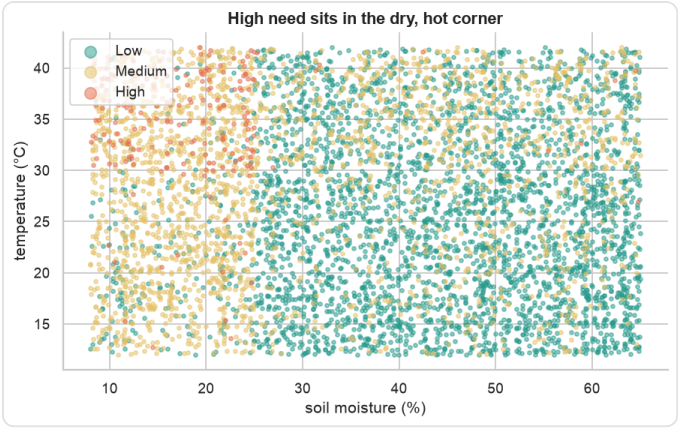


Fig. 13 · **Scatter plot.** High need sits in the dry, hot corner.

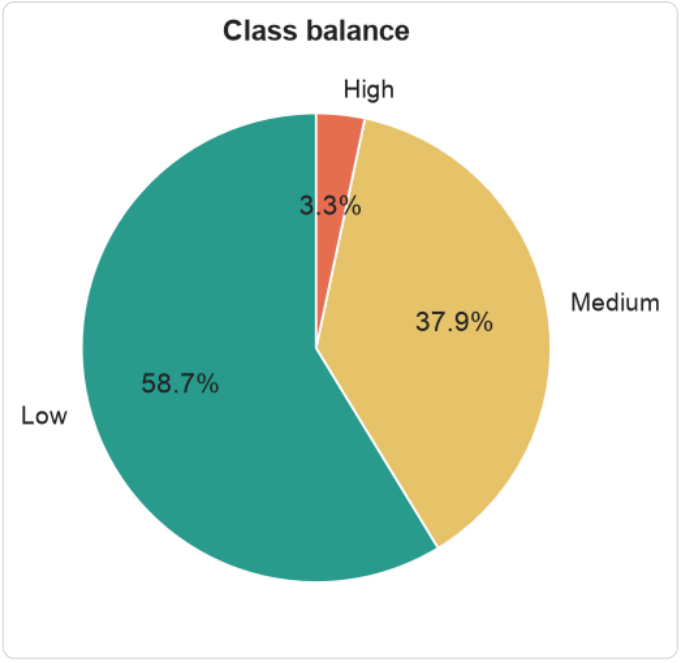


Fig. 14 · **Pie chart** of the class balance.

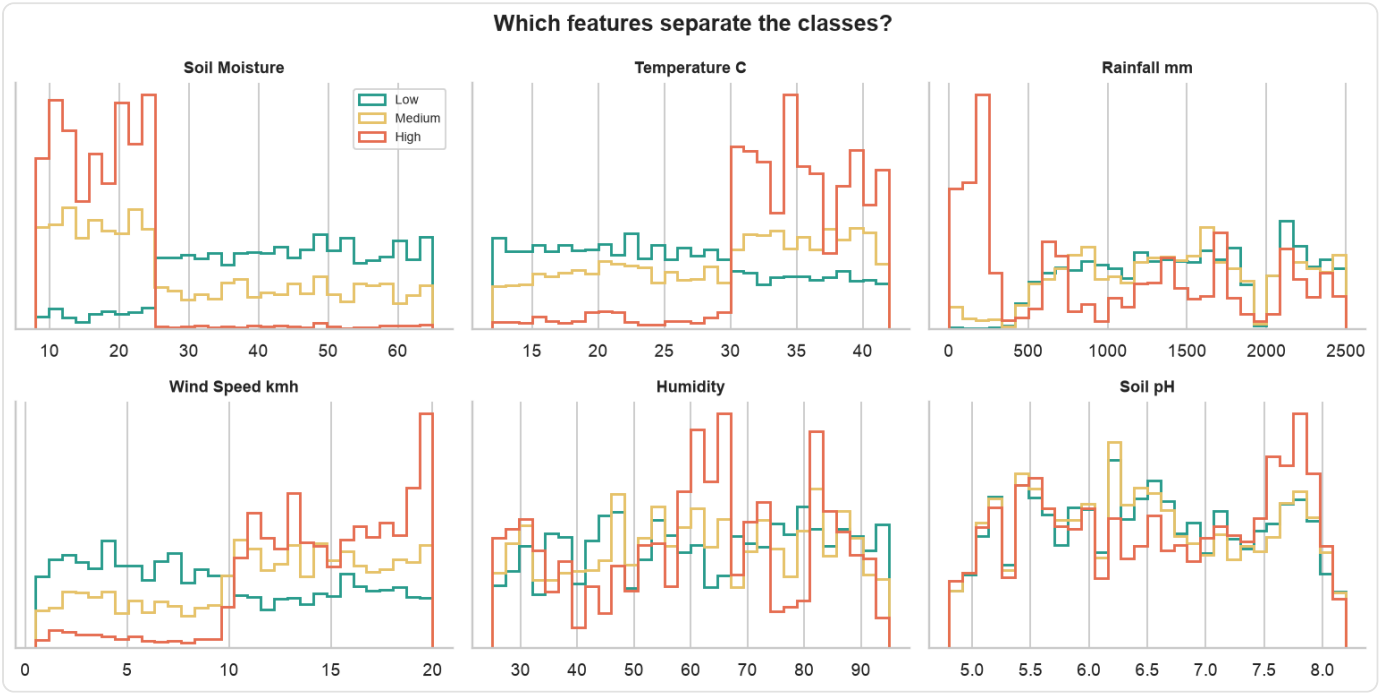


Fig. 15 · **Subplots.** Six features per class side by side: moisture, temperature, rainfall and wind separate the classes; humidity and pH don't.

Exploratory data analysis

Unit III: types of data, loading and inspecting, univariate and bivariate analysis, correlation and multicollinearity, skewness and kurtosis, patterns, anomalies and trends, and how EDA feeds the modelling. `Notebook 03`

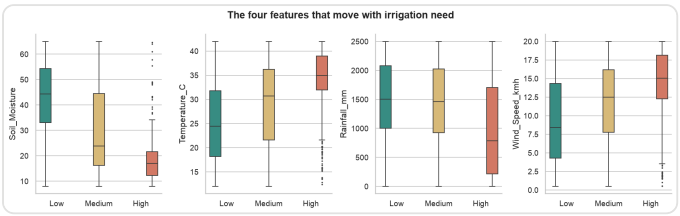


Fig. 16 · **Box plots** of the four features that move with need.

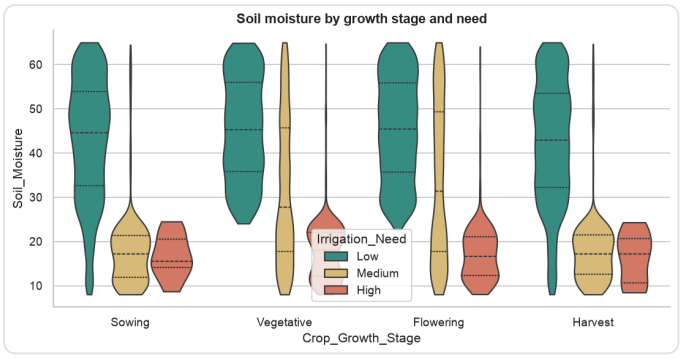


Fig. 17 · **Violin plot.** Moisture by growth stage and need.

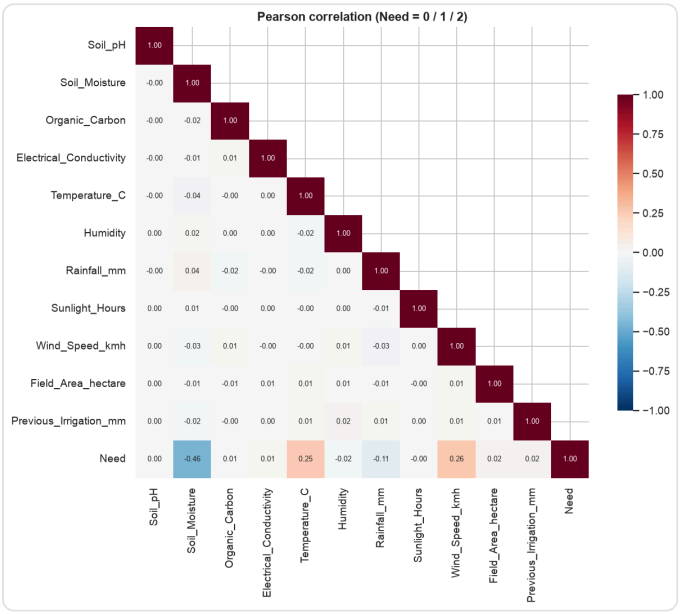


Fig. 18 · **Correlation heatmap.** Need correlates with moisture (−0.46), wind (0.26), temperature (0.25) and rainfall (−0.11); the features hardly correlate with each other.

Multicollinearity

Every feature has a variance inflation factor between 1.000 and 1.006: the features are independent, so linear-model coefficients will be stable. A deliberately added near-copy of soil moisture shows what a real problem looks like: VIF 219 for both columns.

Skewness and kurtosis

All skewness values are within ±0.12. Excess kurtosis is between −0.94 and −1.26, close to a uniform distribution's −1.2. The features are bounded and flat, not bell-shaped.

Anomalies

Neither the IQR rule nor $|z| > 3$ flags a single real value. To show that the rules work, 40 sensor faults were injected into a copy (a stuck probe, Fahrenheit temperatures, negative rainfall). The z-score rule caught 39 and the IQR rule 36. The misses were values that land inside the normal range.

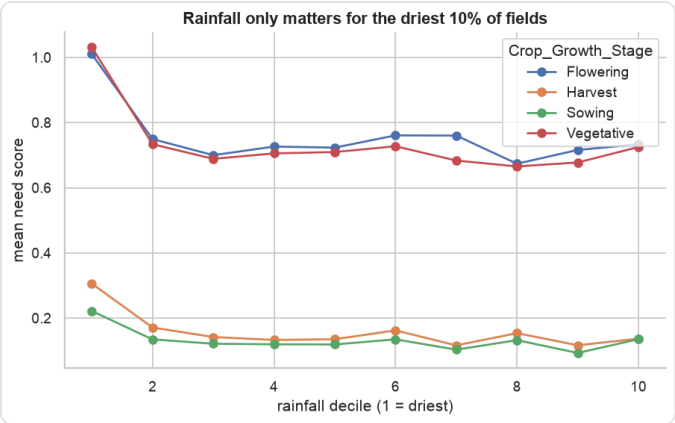


Fig. 19 · **A threshold, not a slope.** Need only jumps in the driest 10% of fields, and the vegetative and flowering stages sit far above the other two.

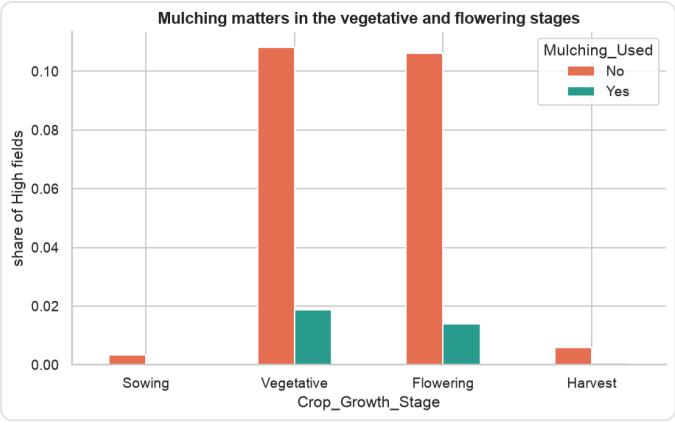


Fig. 20 · **An interaction.** Mulching cuts the High share from about 11% to under 2%, but only in the thirsty stages.

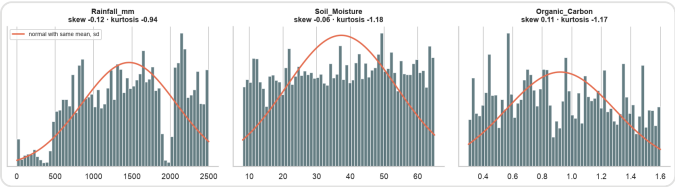


Fig. 21 · **Skewness and kurtosis** against a normal curve with the same mean and SD.

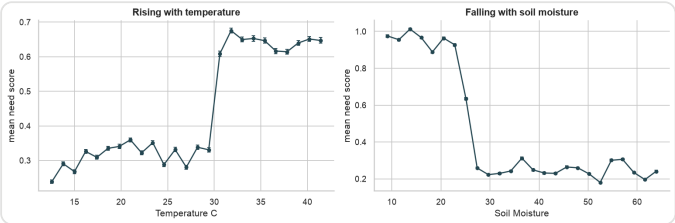


Fig. 22 · **Trends.** Mean need rises with temperature and falls with moisture (95% error bars).

What the EDA decided

FINDING	CONSEQUENCE FOR MODELLING
No missing values or duplicates	no imputation
High is 3.3% of rows	stratified splits; optimise balanced accuracy
Thresholds and a stage × mulching interaction	trees fit naturally; linear models need engineered features
Independent, bounded features	stable coefficients; scaling is safe; no outlier removal

06 · UNIT IV

Linear algebra, probability and calculus

Unit IV: scalars to tensors, eigenvalues and eigenvectors, random variables and distributions, statistical measures, joint, marginal and conditional probability, Bayes' theorem and likelihood, functions, partial derivatives, the gradient and the chain rule.

Notebook 04

Linear algebra

One field is a vector, the table a matrix and a stack of per-region matrices a 5 × 1000 × 11 tensor. A hand-made "dryness" score (one dot product with weights on moisture, temperature, rainfall and wind) already ranks the classes: mean −0.55 for Low, 0.67 for Medium, 2.03 for High. Power iteration written from scratch finds the top eigenvalue of the feature covariance, 1.107249, identical to `np.linalg.eigh`. All eigenvalues lie between 0.93 and 1.11: there is no dominant direction, confirming the independence found in the EDA.

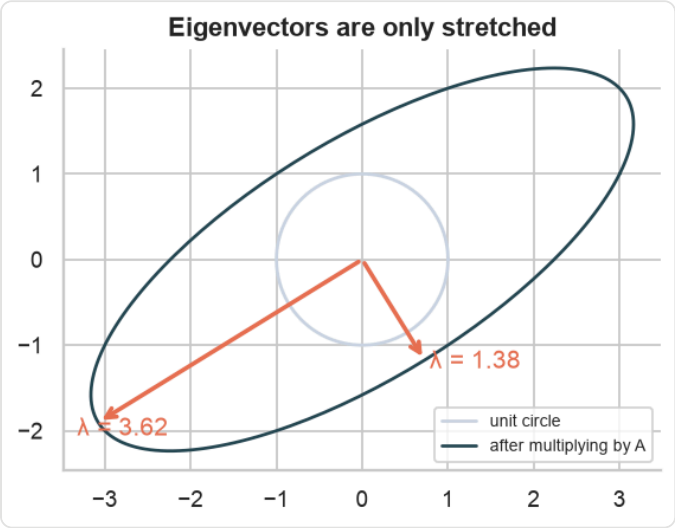


Fig. 23 · **Eigenvectors are only stretched** when the unit circle is multiplied by A.

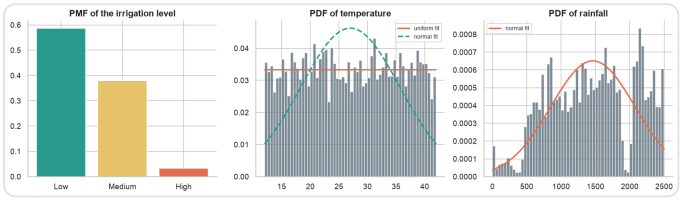


Fig. 24 · **Distributions.** The class PMF, and temperature against uniform and normal fits: the uniform fits.

Probability and Bayes' theorem

QUANTITY	VALUE
P(High) (prior)	0.0333
P(dry | High) (likelihood)	0.6444
P(dry) (evidence)	0.1909
P(High | dry) by Bayes' theorem / counted directly	0.1125 / 0.1125
P(High | no mulch) vs P(High | mulch)	0.0585 vs 0.0079: dependent

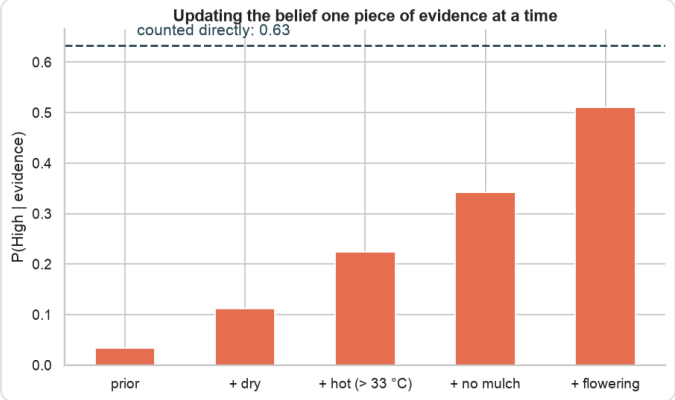


Fig. 25 · **Updating a belief** one piece of evidence at a time (naive assumption) reaches 0.51; the dashed line, counted directly, is 0.63.

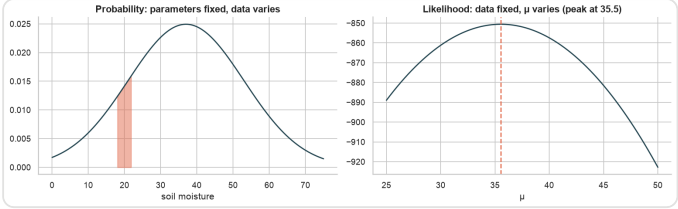


Fig. 26 · **Probability vs likelihood:** the same normal formula read in two directions; the likelihood peaks at the sample mean.

Calculus

The logistic-loss gradient was derived by hand, $\nabla L = X^{\top}(\sigma(Xw) - y)/n$, and compared with central finite differences on all 12 parameters: the largest difference is 8.25×10^{-12}. The chain rule was done step by step for one field and one weight: $dL/dp \times dp/du \times du/dw = -0.130726$, the simplified $(p - y)\cdot x = -0.130726$, and the numerical derivative $= -0.130726$.

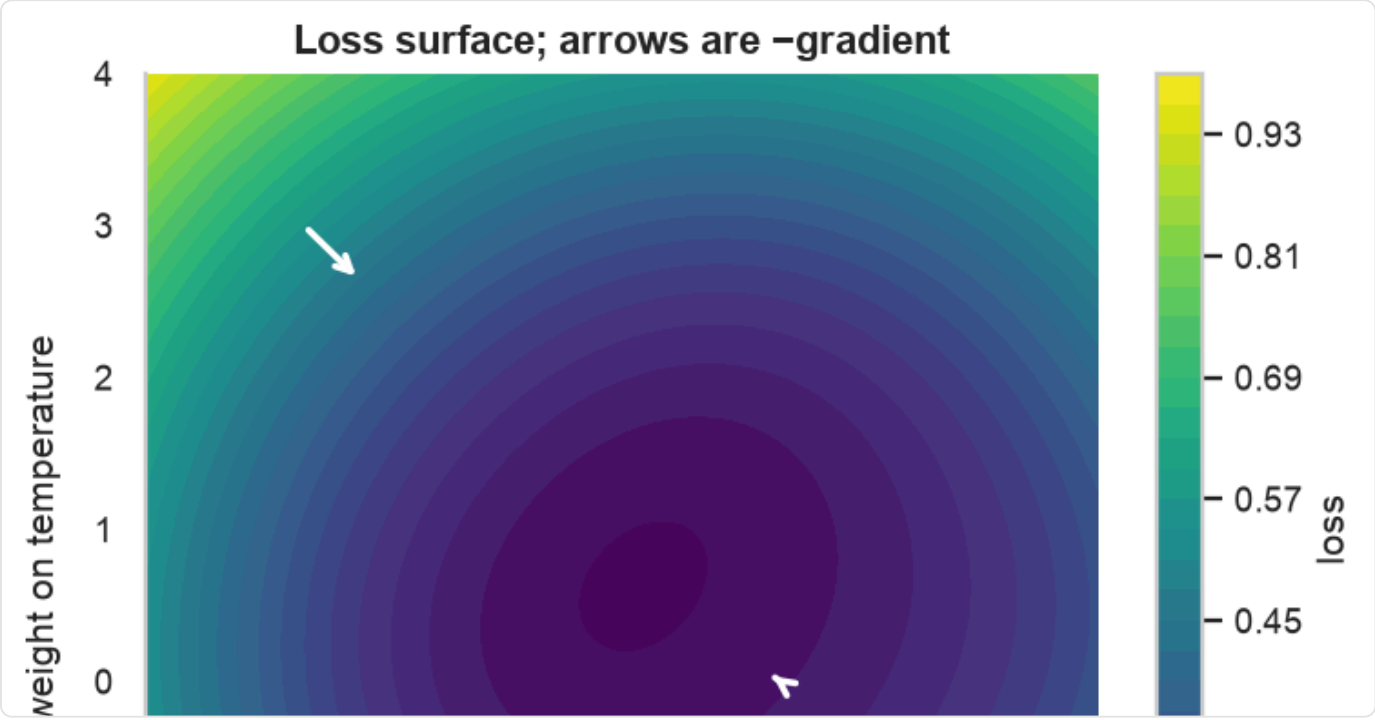


Fig. 27 · **The loss surface** over two weights; the white arrows (negative gradient) point to low loss: negative weight on moisture, positive on temperature.

07 · UNIT V

Statistics for machine learning and optimisation

Unit V: sampling and population, hypothesis testing, confidence intervals, correlation, maximum likelihood, loss functions, convex and non-convex functions, minima and saddle points, gradient descent and its learning rate and variants, and the Momentum, RMSProp and Adam optimisers. `Notebook 05`

Sampling, tests and intervals

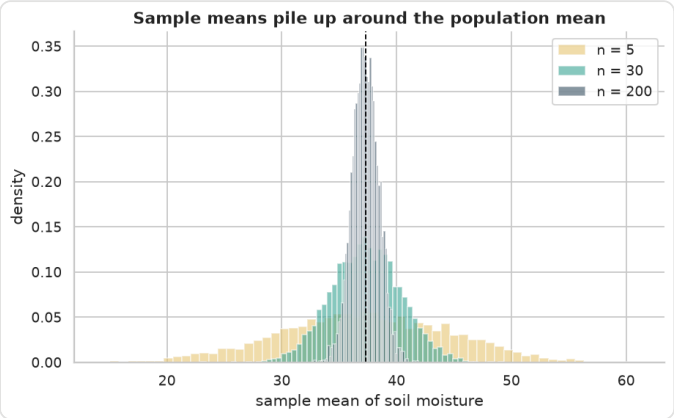


Fig. 28 · **The central limit theorem, measured.** Sample means of a flat feature are bell-shaped; their spread matches σ/√n (7.38 vs 7.32 at n = 5; 1.154 vs 1.158 at n = 200).

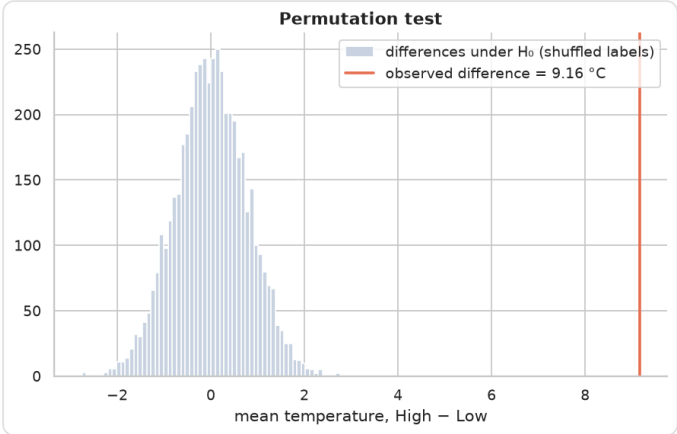


Fig. 29 · **Permutation test.** None of 5,000 label shuffles comes near the real 9.16 °C difference between High and Low fields.

TEST	RESULT
Welch t-test, temperature High vs Low	t = 17.92, p = 5.6×10^{-41} → reject H_0
χ^2 test, mulching vs need	χ^2 = 354.8, p = 9.0×10^{-78} → dependent
Welch t-test, soil pH High vs Low	p = 0.994 → keep H_0 (no difference)
Type I error over 2,000 A/A tests at α = 0.05	0.053
95% CI for mean temperature of High fields (t / bootstrap / `conviction`)	[33.63, 35.53] / [33.61, 35.47] / [33.63, 35.50]; all contain the true 34.57
`conviction`**: P(High − Low difference > 9 °C)**	0.631, so not decided at 95% confidence

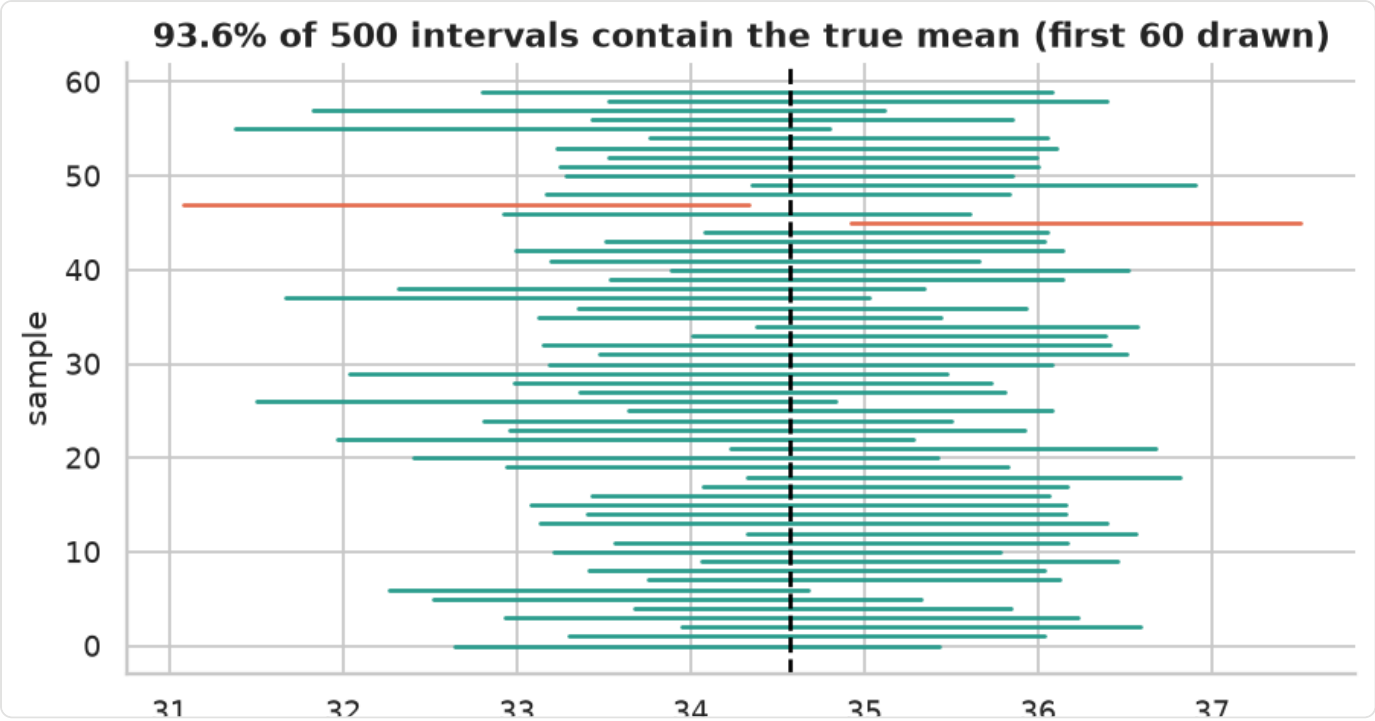


Fig. 30 · **What 95% means.** 500 t-intervals from samples of 60 fields; 93.6% contain the true mean.

Estimation and losses

For temperature, the closed-form MLE (μ = 27.1160, σ = 8.5716) and a Nelder–Mead minimisation of the negative log-likelihood (27.1159, 8.5716) agree. With one broken reading of 999 among 100 moisture values, the best constant under MSE is the mean, 43.26, while MAE gives the median, 29.7: one outlier moves only the first.

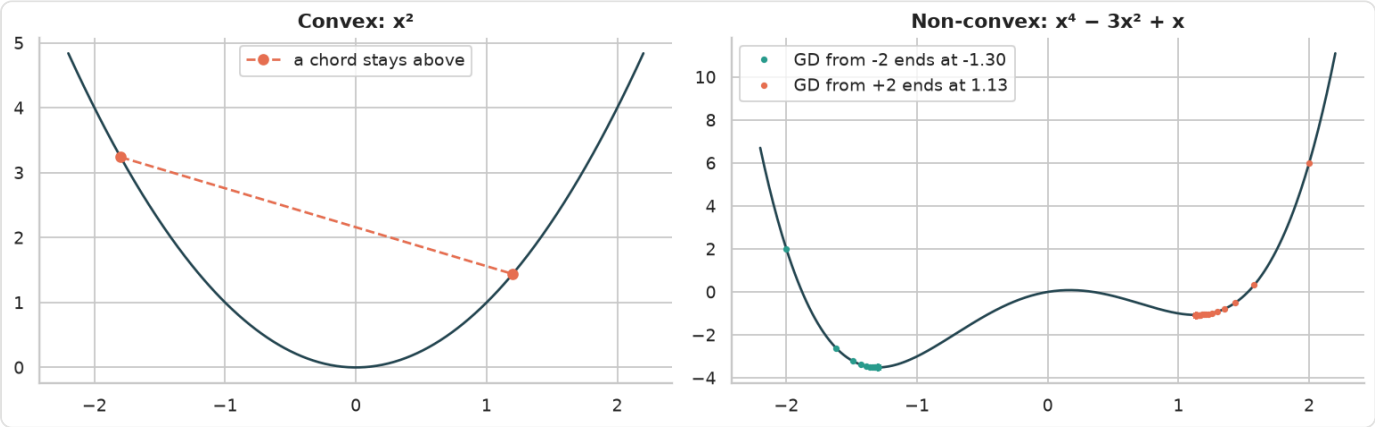


Fig. 31 · **Convex vs non-convex.** On the non-convex curve, GD ends in a different minimum depending on where it starts.

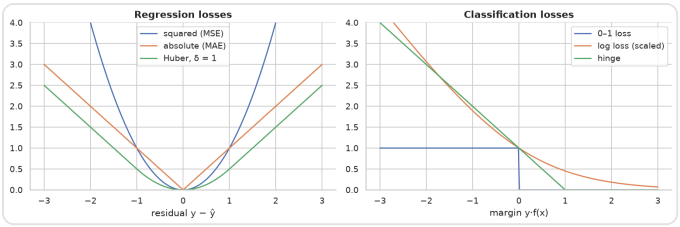


Fig. 32 · **Loss functions:** MSE, MAE and Huber; 0–1, log and hinge loss against the margin.

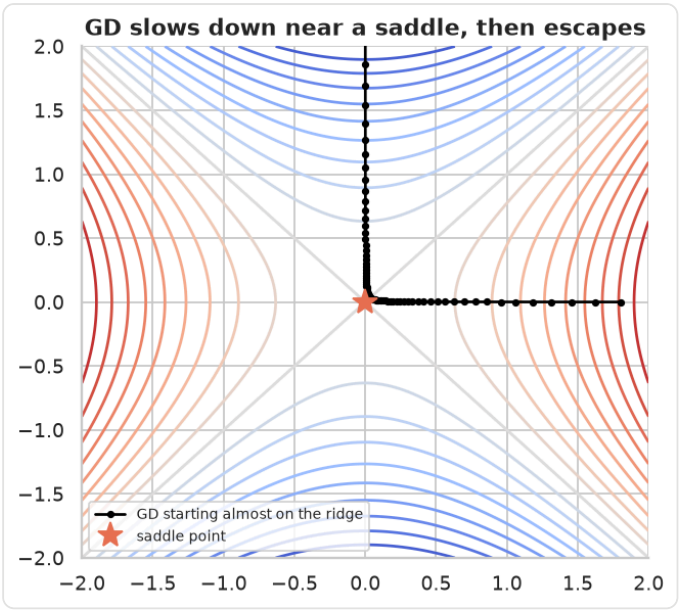


Fig. 33 · **A saddle point:** Hessian eigenvalues −2 and 2; gradient descent stalls near it, then escapes.

Gradient descent and the optimisers

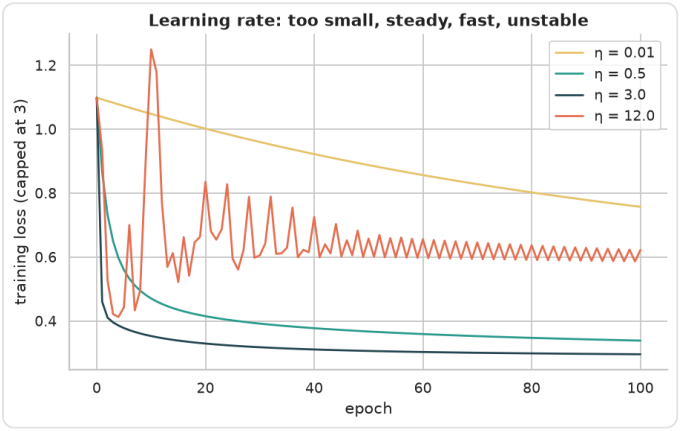


Fig. 34 · **The learning rate.** Too small is slow; 3.0 is fast; 12 overshoots and bounces.

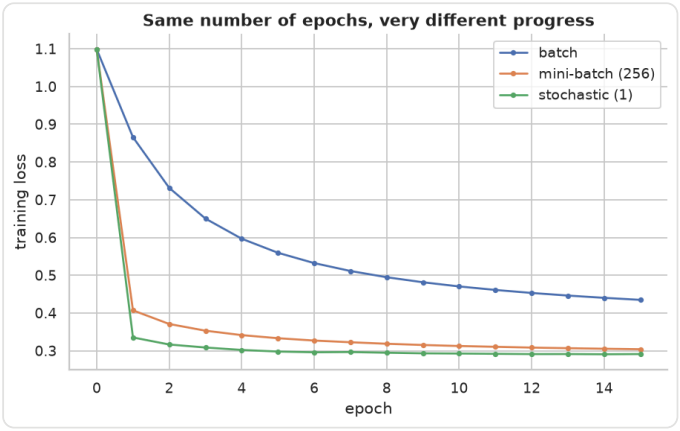


Fig. 35 · **Batch, mini-batch and stochastic GD** after the same 15 epochs.

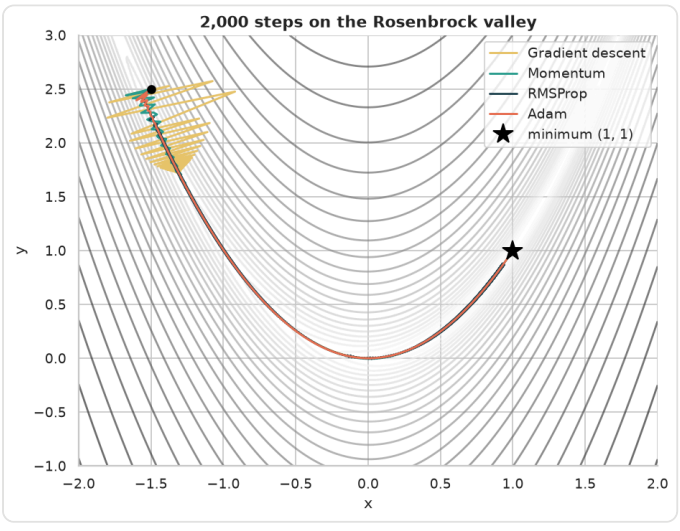


Fig. 36 · **Rosenbrock valley, 2,000 steps.** Adam ends 3 × 10^{-6} from the minimum and Momentum 5 × 10^{-5}; RMSProp bounces along the floor (0.14 away); GD crawls (0.58).

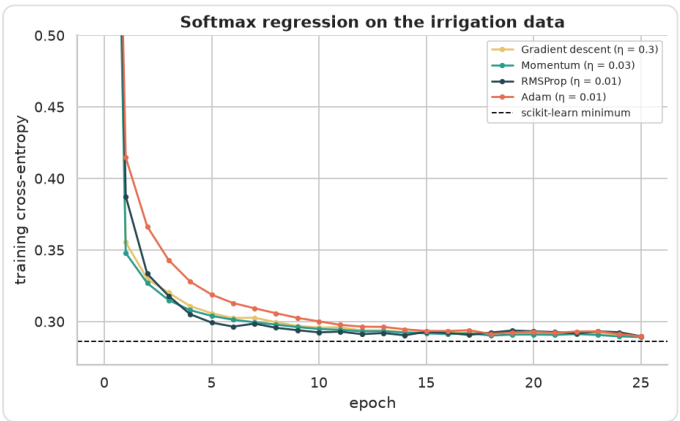


Fig. 37 · **Softmax regression on the irrigation data.** With tuned learning rates all four reach 0.289–0.290 against scikit-learn's 0.2865.

The honest conclusion: on a convex, well-scaled problem the learning rate matters more than the optimiser; the adaptive methods pull ahead on badly conditioned landscapes like Rosenbrock.

08 · UNIT VI

Statistical learning theory

Unit VI: well-posed learning problems, the components of a learning system, the statistical learning framework, empirical risk minimisation, overfitting and generalisation, inductive bias, prior knowledge, PAC learning, sample complexity and the VC dimension, the No Free Lunch theorem, and choosing algorithms with Occam's razor. Notebook 06

With the 630,000 fields standing in for the unknown distribution, the true risk of a hypothesis can be computed exactly. For most of the unit the task is binary: does the field need water (Medium or High, 41.3%)? The first hypothesis class is the 400 decision stumps on soil moisture; the best of them ("needs water when moisture < 25.2%") has a true risk of 0.222.

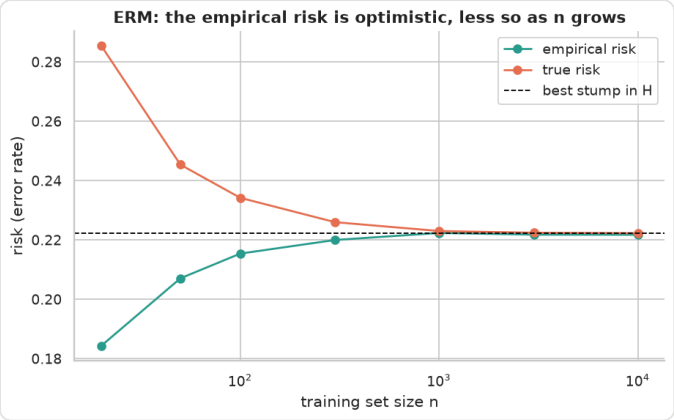


Fig. 38 · **ERM.** With 20 examples the empirical risk (0.18) is far below the true risk (0.29); by 10,000 the gap is 0.0005.

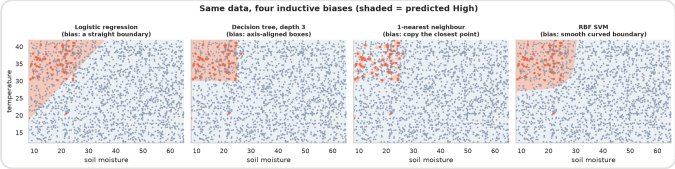


Fig. 39 · **Inductive bias.** Four learners, one dataset, four different answers on every unseen point.

Overfitting needs noise

TRAINING LABELS	N	DEPTH-6 TREE, TEST RISK	UNLIMITED TREE, EMPIRICAL / TEST RISK
clean	40,000	0.013	0.000 / 0.025
20% flipped	1,000	0.168	0.000 / 0.242
20% flipped	40,000	0.015	0.000 / 0.235

With clean labels the tree that memorises does no harm; with noisy labels it memorises the noise and more data doesn't help, while the restricted tree keeps improving.

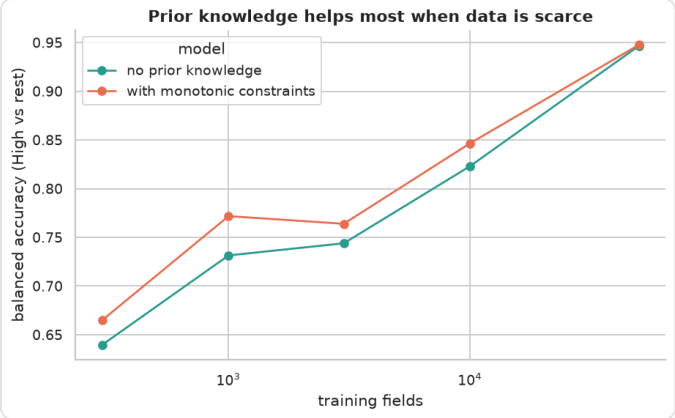


Fig. 40 · **Prior knowledge.** Monotonic constraints ("wetter never needs more water") add 2–4 points of balanced accuracy below 10,000 fields, and 0.1 points at 50,000.

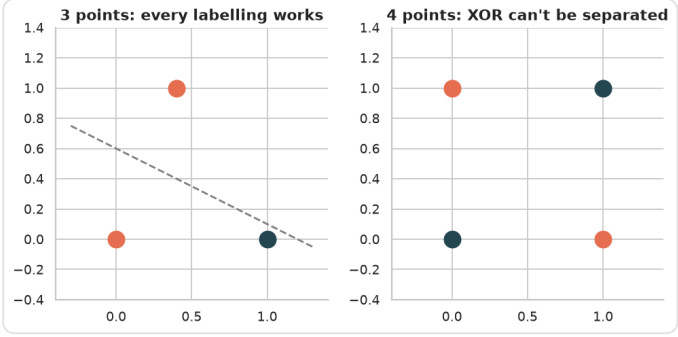


Fig. 41 · **VC dimension.** Lines shatter 3 points but not 4 (the XOR labellings), checked with a linear program.

PAC learning, tested

For $|H| = 400$, $\varepsilon = 0.02$ and $\delta = 0.05$, the agnostic bound asks for $m = 12{,}101$ examples. In 300 independent samples of that size, the largest gap between empirical and true risk over all 400 stumps exceeded ε in **0** of them (the worst gap seen was 0.0165). Then a practical guarantee: my library `conviction`'s `RiskGuard`, calibrated on 18,000 fields, certified that among fields auto-labelled "not High" at most 0.5% are wrong, with 95% confidence. It auto-labels 89.9% of 18,000 *unseen* fields, with an error rate of 0.365%.

No Free Lunch and choosing algorithms

On a world of 8 inputs, training on 4 and averaging over all 256 possible labellings of the rest, every learner scores exactly 0.5 off the training set: always-0, majority, 1-NN, a decision tree and an anti-learner. In practice the winner depends on the data: k-NN wins on moons (0.941) and circles (0.991), logistic regression on a linear problem (0.919) and the tree on the irrigation data (0.937). The one-standard-error rule picks depth 7, which is also the best depth here.

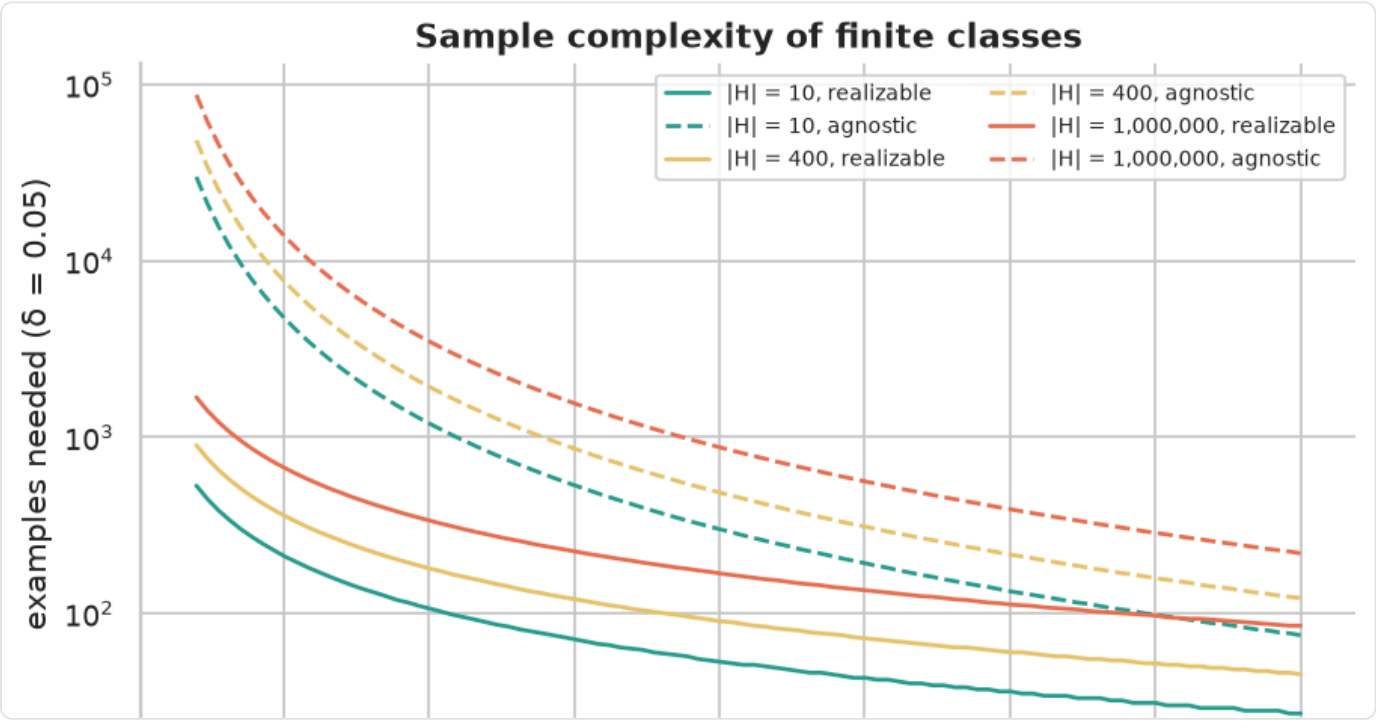


Fig. 42 · **Sample complexity** of finite classes, realisable and agnostic.

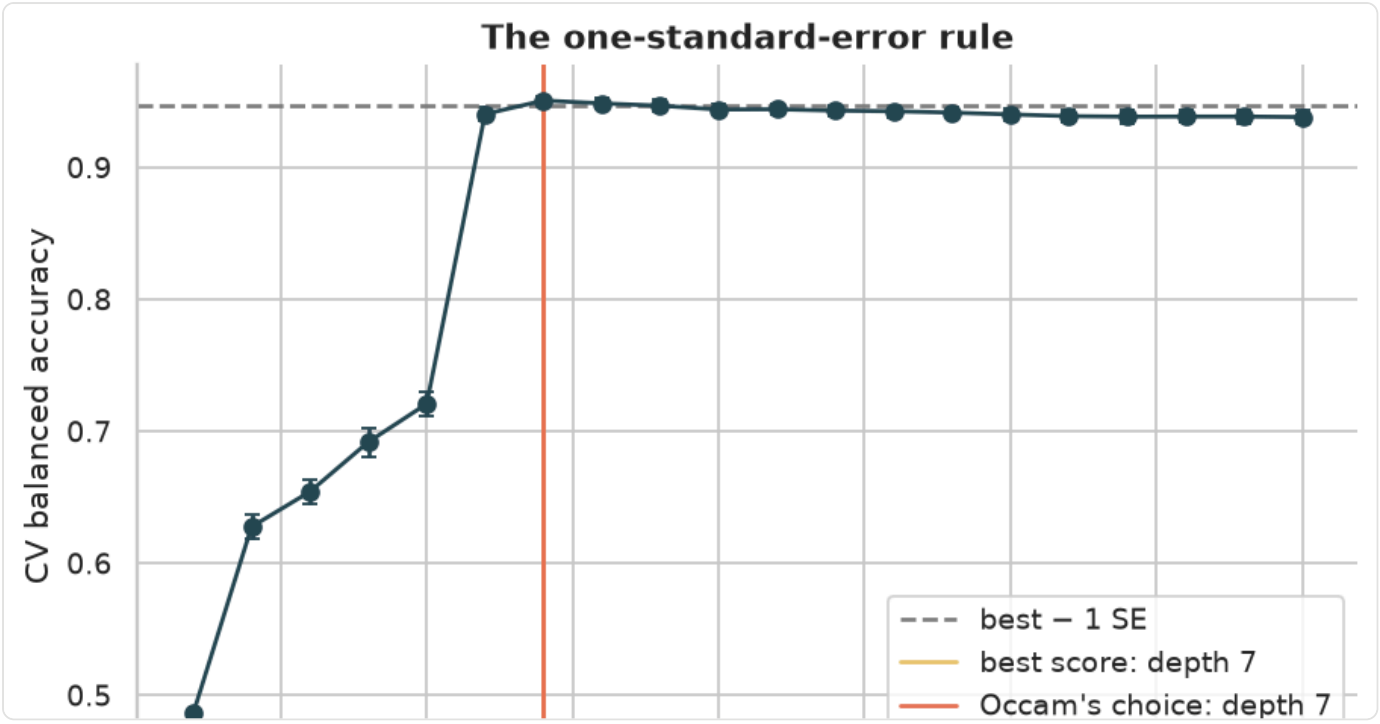


Fig. 43 · **Occam's razor:** the one-standard-error rule on tree depth.

09 · SYLLABUS MAPPING

Every topic of the syllabus

Generated from `tools/syllabus.py`, the same file that builds the course map on the website, so the two cannot disagree. "Where" gives the notebook and section.

Unit I: Machine Learning Foundation 12 topics

Notebook `01_foundations_numpy.ipynb` · Machine learning foundations and NumPy

SYLLABUS TOPIC	WHERE
Introduction to machine learning	01 § 1
AI vs machine learning vs data science	01 § 2
Supervised, unsupervised and reinforcement learning	01 § 3
Parametric vs non-parametric models	01 § 4
Generative vs discriminative models	01 § 4
Machine learning workflow	01 § 5
Train–test split, overview of evaluation metrics	01 § 6
Overfitting and underfitting	01 § 6
Bias–variance trade-off (measured with 30 bootstrap fits)	01 § 6
Applications and use cases	01 § 7
NumPy: why NumPy, creating arrays, operations	01 § 8
Broadcasting rule, matrix operations, random module	01 § 8

Unit II: Pandas for Data Handling 7 topics

Notebook `02_pandas_matplotlib.ipynb` · Pandas for data handling

SYLLABUS TOPIC	WHERE
Pandas Series and DataFrame	02 § 1
Reading and writing CSV files	02 § 2
Inspecting datasets	02 § 3
Selection: loc vs iloc; boolean filtering	02 § 4
Line, bar, histogram, scatter and pie charts	02 § 5
Customising plots	02 § 6
Subplots	02 § 7

Unit III: Exploratory Data Analysis 8 topics

Notebook `03_eda.ipynb` · Exploratory data analysis

SYLLABUS TOPIC	WHERE
Types of data (quantitative, qualitative)	03 § 1

SYLLABUS TOPIC	WHERE
Loading data and initial inspection	03 § 2
Univariate analysis: histogram, box plot, count plot	03 § 3
Bivariate analysis: scatter, line, violin plots	03 § 4
Correlation heatmaps and multicollinearity (VIF)	03 § 5
Skewness and kurtosis	03 § 6
Patterns, anomalies (outliers) and trends	03 § 7
EDA workflow integration	03 § 8

Unit IV: Linear Algebra and Calculus 9 topics

Notebook `04_linear_algebra_probability_calculus.ipynb` · Linear algebra, probability and calculus

SYLLABUS TOPIC	WHERE
Data structures: scalar, vector, matrix, tensor	04 § 1.1
Eigenvalues and eigenvectors (power iteration from scratch)	04 § 1.2
Random variables and probability distributions	04 § 2.1–2.2
Statistical measures	04 § 2.3
Types of probability: joint, marginal, conditional	04 § 2.4
Bayes' theorem and inference	04 § 3.1
Likelihood vs probability	04 § 3.2
Functions, partial derivatives and the gradient	04 § 4.1–4.3
The chain rule	04 § 4.4

Unit V: Statistics for ML 11 topics

Notebook `05_statistics_optimisation.ipynb` · Statistics for ML and optimisation

SYLLABUS TOPIC	WHERE
Sampling and population	05 § 1.1
Hypothesis testing	05 § 1.2
Confidence intervals	05 § 1.3
Correlation	05 § 1.4
Maximum likelihood estimation	05 § 2.1
Loss (cost) functions	05 § 2.2
Convex vs non-convex functions	05 § 3.1
Minima and saddle points	05 § 3.2
Gradient descent and the learning rate	05 § 4.1–4.2
Variants of gradient descent (batch, stochastic, mini-batch)	05 § 4.3

SYLLABUS TOPIC	WHERE
Momentum, RMSProp and Adam (from scratch)	05 § 5

Unit VI: Statistical Learning Theory 10 topics

Notebook `06_learning_theory.ipynb` · Statistical learning theory

SYLLABUS TOPIC	WHERE
Well-posed learning problems (T-P-E)	06 § 1
Components of a learning system	06 § 2
Statistical learning framework	06 § 3
Empirical risk minimisation, overfitting and generalisation	06 § 4
Inductive bias	06 § 5
Role of prior knowledge	06 § 6
PAC learning	06 § 7
Sample complexity: finite hypothesis space and VC dimension	06 § 8
No Free Lunch theorem	06 § 9
Choosing algorithms; Occam's razor	06 § 10

The important functions

Adam

optim.py · lines 47–63

```
class Adam:
    name = "Adam"

    def __init__(self, lr=0.01, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr, self.beta1, self.beta2, self.eps = lr, beta1, beta2, eps
        self.m = self.v = None
        self.t = 0

    def step(self, w, g):
        if self.m is None:
            self.m, self.v = np.zeros_like(w), np.zeros_like(w)
        self.t += 1
        self.m = self.beta1 * self.m + (1 - self.beta1) * g
        self.v = self.beta2 * self.v + (1 - self.beta2) * g**2
        m_hat = self.m / (1 - self.beta1**self.t)  # bias correction: m and v start at 0
        v_hat = self.v / (1 - self.beta2**self.t)
        return w - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

Every optimiser has the same interface, `step(w, g)`, so they are interchangeable in `minimize()` and `train_softmax()`. Adam keeps a running mean of the gradient (the momentum term *m*) and of its square (the RMSProp term *v*).

Both start at zero, so early on they are biased towards zero; dividing by $1 - \beta^t$ corrects that. Without it the first steps would be far too small. The test suite checks that all four optimisers find the minimum of a bowl and that Adam beats plain GD on Rosenbrock.

Softmax regression and its gradient

optim.py · lines 104–110

```
def cross_entropy(W, X, Y):
    """Mean cross-entropy loss and its gradient for softmax regression.
    X has a bias column, Y is one-hot."""
    P = softmax(X @ W)
    loss = -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))
    grad = X.T @ (P - Y) / len(X)
    return loss, grad
```

The loss of a softmax model is the mean cross-entropy; its gradient with respect to the weight matrix has the compact form $X^T(P - Y)/n$. Notebook 04 derives the same formula for the binary case through the chain rule, and `tests/test_optim.py` checks it against finite differences.

`softmax()` subtracts each row's maximum before exponentiating; without that, large scores overflow to infinity.

optim.py · lines 113–129

```
def train_softmax(X, y, optimizer, epochs=30, batch_size=None, seed=0):
    """batch_size=None is full-batch GD, 1 is SGD, anything else is mini-batch."""
    rng = np.random.default_rng(seed)
    X = np.hstack([np.ones((len(X), 1)), X])
    k = int(y.max()) + 1
    Y = np.eye(k)[y]
    W = np.zeros((X.shape[1], k))
    bs = len(X) if batch_size is None else batch_size
    history = [cross_entropy(W, X, Y)[0]]
    for _ in range(epochs):
        order = rng.permutation(len(X))
        for start in range(0, len(X), bs):
            idx = order[start:start + bs]
            _, g = cross_entropy(W, X[idx], Y[idx])
            W = optimizer.step(W, g)
        history.append(cross_entropy(W, X, Y)[0])
    return W, np.array(history)
```

One function covers all three gradient-descent variants: `batch_size=None` is batch GD, 1 is SGD, anything else is mini-batch; the data is reshuffled every epoch.

Shattering and the VC dimension

theory.py · lines 26–32

```
def linearly_separable(X, y):
    """Is there w, b with y_i (w.x_i + b) >= 1 for all i? Solved as an LP feasibility problem."""
    X = np.asarray(X, dtype=float)
    s = np.where(np.asarray(y) > 0, 1.0, -1.0)
    A = -s[:, None] * np.hstack([X, np.ones((len(X), 1))])
    res = linprog(np.zeros(X.shape[1] + 1), A_ub=A, b_ub=-np.ones(len(X)), bounds=[(None, None)] * (X.shape[1] + 1))
    return res.status == 0
```

"Can a line separate these labels?" is a linear-programming feasibility question: find w and b such that every point is on the right side with margin 1. If the LP is feasible, the labelling is realisable.

`shattered()` tries every one of the 2^n labellings; the class shatters the points only if all of them are feasible. For four corners of a square, the two XOR labellings fail, so lines in the plane have VC dimension 3.

theory.py · lines 35–40

```
def shattered(X):
    """Can lines in the plane realise every one of the 2^n labelings of these points?"""
    X = np.asarray(X, dtype=float)
    failed = [labels for labels in product([0, 1], repeat=len(X))
              if len(set(labels)) > 1 and not linearly_separable(X, labels)]
    return len(failed) == 0, failed
```

PAC sample size and ERM

theory.py · lines 13–18

```
def pac_sample_size(h_size, eps, delta, realizable=True):
    """Training examples needed so that, with probability >= 1 - delta, the learned
    hypothesis has true error <= eps (realizable) or within eps of the best (agnostic)."""
    if realizable:
        return int(np.ceil((np.log(h_size) + np.log(1 / delta)) / eps))
    return int(np.ceil((np.log(h_size) + np.log(2 / delta)) / (2 * eps**2)))
```

The two classic finite-class bounds: realisable, $(\ln|H| + \ln(1/\delta))/\varepsilon$, and agnostic, $(\ln|H| + \ln(2/\delta))/(2\varepsilon^2)$. Notebook 06 uses the agnostic one because no stump classifies every field correctly, and checks it on 300 samples.

theory.py · lines 49–58

```
def erm_stump(x, y, thresholds=None):
    """Empirical risk minimisation: pick the stump with the lowest training error."""
    thresholds = np.unique(x) if thresholds is None else thresholds
    best = (np.inf, None, None)
    for t in thresholds:
        for sign in (1, -1):
            risk = np.mean(stump_predict(x, t, sign) != y)
            if risk < best[0]:
                best = (risk, t, sign)
    return best
```

The bootstrap

stats.py · lines 17–22

```
def bootstrap_ci(values, stat=np.mean, n_boot=2000, level=0.95, seed=0):
    rng = np.random.default_rng(seed)
    values = np.asarray(values)
    boots = np.array([stat(rng.choice(values, len(values))) for _ in range(n_boot)])
    lo, hi = np.quantile(boots, [(1 - level) / 2, 1 - (1 - level) / 2])
    return stat(values), lo, hi
```

Resample the data with replacement, recompute the statistic, and read the interval off the quantiles of the resampled values. The test suite checks that it covers the true mean in at least 50 of 60 repeated experiments.

What was checked

Three layers: unit tests of the package, checks inside the notebooks that compare each from-scratch result with an independent answer, and a full execution of every notebook on a fresh kernel.

1. Unit tests (pytest)

TEST	CHECKS	RESULT
`test_optim.py`	every optimizer finds the minimum of a bowl[opt0]	Pass
`test_optim.py`	every optimizer finds the minimum of a bowl[opt1]	Pass
`test_optim.py`	every optimizer finds the minimum of a bowl[opt2]	Pass
`test_optim.py`	every optimizer finds the minimum of a bowl[opt3]	Pass
`test_optim.py`	gd diverges when the learning rate is too big	Pass
`test_optim.py`	rosenbrock gradient matches finite differences	Pass
`test_optim.py`	adam gets closer to rosenbrock minimum than plain gd	Pass
`test_optim.py`	softmax gradient is correct	Pass
`test_optim.py`	softmax regression learns separable blobs	Pass
`test_stats_theory.py`	mle normal matches numpy population formulas	Pass
`test_stats_theory.py`	mle minimises the negative log likelihood	Pass
`test_stats_theory.py`	bootstrap interval contains the true mean most of the time	Pass
`test_stats_theory.py`	huber is quadratic inside and linear outside	Pass
`test_stats_theory.py`	pac bounds grow with hypotheses and shrink with eps	Pass
`test_stats_theory.py`	lines shatter three points but not four	Pass
`test_stats_theory.py`	erm stump finds a perfect threshold	Pass

```
$ pytest -q
................  [100%]
16 passed in 1.62s
```

2. Checks inside the notebooks

WHAT	COMPARED WITH	RESULT
Hand-derived logistic gradient	central finite differences, 12 parameters	**max difference 8.25×10^{-12}**
Chain rule, one weight	simplified formula and numerical derivative	**all −0.130726**
Power iteration	`np.linalg.eigh`	**λ = 1.107249 both, |cos| = 1.0**
Normal MLE (closed form)	Nelder–Mead on the negative log-likelihood	**agree to 4 decimals**
Bayes' theorem	direct counting	**0.1125 both**
Four optimisers	scikit-learn L-BFGS on the same loss	**within 0.004**

WHAT	COMPARED WITH	RESULT
PAC bound	300 samples of the bound's size	**0 failures (5% allowed)**
`RiskGuard` **certificate**	18,000 unseen fields	**0.365% error, promised ≤ 0.5%**
Type I error at α = 0.05	2,000 A/A tests	**0.053**

3. Notebook execution

Every notebook was executed from top to bottom on a fresh kernel with `jupyter nbconvert --execute`, on Python 3.12 with NumPy 2.5, pandas 3.1 and scikit-learn 1.9, with no errors. Reading the outputs against the text found and fixed several wrong claims before they were published:

- The first bias–variance experiment used a target the features can't predict; it was replaced with one they can.
- A pandas 3 change made `scipy.stats.zscore` return an array, so a per-column outlier count silently became a total. It now uses a pandas z-score.
- The first optimiser comparison used untuned learning rates and suggested an order that a fair sweep didn't support.
- The overfitting demo showed no overfitting at all on the clean, nearly noise-free labels, so the noisy-label version was added.
- The first "linear" toy dataset turned out not to be linear.

12 · DEPLOYMENT AND CONCLUSION

Online, and what I learned

Deployment

Website	https://lushadowx.github.io/Groundwork/: overview, results, course map and every notebook as HTML
Source code	https://github.com/LuShadowX/Groundwork
Run locally	`pip install -e ".[dev]"`, download the competition data into `data/`, then `pytest` and `jupyter lab notebooks/`
Run on Kaggle	open a notebook with the competition attached; the first cell installs the package from GitHub
Rebuild the site	`python tools/build_site.py`

The competition data is not in the repository: Kaggle's rules don't allow redistributing it.

What I learned

- **A theorem is easier to trust after you've watched it hold.** Seeing the PAC bound survive 300 samples and the No Free Lunch average come out at exactly 0.5 taught me more than the proofs alone.
- **Check every claim against the output.** Several sentences I wrote before running the code turned out to be wrong; the data has the final say.
- **Real data is boring in useful ways.** No missing values, no outliers, independent features: those are findings, and they changed which methods were worth using.
- **Optimisers are simple; learning rates are not.** Four optimisers fit in 60 lines; choosing their learning rates fairly took a sweep.
- **Imbalance changes everything downstream.** 3.3% High fields is why balanced accuracy matters, and why my first Kaggle model stopped at 0.960 (fixed in the CSE274 project).

Possible extensions

- A learning-rate schedule for the optimisers, to close the last gap to the true minimum.
- Conformal prediction sets for all three classes, not just "not High".
- The same theory experiments on real, noisier field data.